REPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique

Concours Nationaux d'Entrée
aux Cycles de Formation d'Ingénieurs
Session 2018

الجمهورية التونسية
وزارة التعليم العالي والبحث العلمي

المناظرات الوطنية للدخول
إلى مراحل تكوين المهندسين
دورة 2018

Concours Mathématiques et Physique, Physique et Chimie et Technologie
Epreuve d'Informatique

Date : Vendredi 1 Juin 2018 Heure : 11 H Durée : 2 H Nombre de pages : 6

Barème : Problème 1 : 14 points
Problème 2 : 6 points

DOCUMENTS NON AUTORISES
L'USAGE DES CALCULATRICES EST INTERDIT
Il FAUT RESPECTER IMPERATIVEMENT LES NOTATIONS DE L'ENONCE

PROBLEME 1

Une image en niveaux de gris est un ensemble de pixels organisés en lignes et colonnes. Chaque pixel est défini par une valeur représentant un niveau de gris (ou encore ton) parmi 256 valeurs possibles entre le noir et le blanc (0 : noir, 255 : blanc).
Binariser une image consiste à la partitionner en deux parties : une partie sombre et une partie claire selon un seuil de niveau de gris.

N.B : Les parties I et II sont indépendantes.

Partie I

Une image peut être binarisée selon un seuil optimal déterminé par une méthode de seuillage. Dans cette partie, on propose d'implémenter quelques étapes d'une méthode de seuillage qui consiste à déterminer un couple de paramètres intervenant dans le calcul du seuil optimal.

Description de la méthode :

Soit une image en niveaux de gris représentée par une matrice M à n lignes et m colonnes, où $M_{i,j} \in [0,255]$ est le niveau de gris associé au pixel d'indices (i, j) $(0 \le i \le n-1$ et $0 \le j \le m-1)$.

La méthode consiste à :

- Déterminer la matrice, notée V, à n lignes et m colonnes nommée matrice des moyennes des voisinages, où chaque élément $V_{i,j}$ $(0 \le i \le n-1$ et $0 \le j \le m-1)$ a pour valeur :
  - $M_{i,j}$, pour $i=0$ ou $j=0$ ou $i=n-1$ ou $j=m-1$;
  - la partie entière de la moyenne des niveaux de gris de l'élément $M_{i,j}$ et de ses 8 voisins immédiats, pour $0<i<n-1$ et $0<j<m-1$.

- Déterminer une matrice carrée d'ordre 256, notée ***H*** et nommée matrice histogramme, où chaque élément $H_{i,j}$ ($0 \leq i \leq 255$ et $0 \leq j \leq 255$) représente le nombre des pixels ayant un niveau de gris égal à ***i*** et une moyenne des niveaux de gris du voisinage égale à ***j***.
- Déterminer la matrice colonne, notée *Moy*, définie par :

$$Moy = \begin{pmatrix} \sum_{i=0}^{255}\sum_{j=0}^{255} i.H_{i,j} \\ \sum_{i=0}^{255}\sum_{j=0}^{255} j.H_{i,j} \end{pmatrix}$$

- A partir de la matrice ***H***, déterminer parmi tous les couples (s,t) possibles, le couple optimal dont la valeur de la trace de la matrice variance correspondante, ci-dessous définie, est maximale.

  Le couple optimal est déterminé comme suit :

  - Pour chaque couple d'indices (s,t) de H tels que $0 \leq s < 255$ et $0 \leq t < 255$:
    - Calcul des probabilités P_0 et P_1 définies par :

$$P_0 = \frac{1}{m\,n}\sum_{i=0}^{s}\sum_{j=0}^{t} H_{i,j} \qquad P_1 = \frac{1}{m\,n}\sum_{i=s+1}^{255}\sum_{j=t+1}^{255} H_{i,j}$$

    - Construction des matrices colonnes Moy_0 et Moy_1 définies par :

$$Moy_0 = \begin{pmatrix} \frac{1}{P_0}\sum_{i=0}^{s}\sum_{j=0}^{t} i.H_{i,j} \\ \frac{1}{P_0}\sum_{i=0}^{s}\sum_{j=0}^{t} j.H_{i,j} \end{pmatrix} \quad Moy_1 = \begin{pmatrix} \frac{1}{P_1}\sum_{i=s+1}^{255}\sum_{j=t+1}^{255} i.H_{i,j} \\ \frac{1}{P_1}\sum_{i=s+1}^{255}\sum_{j=t+1}^{255} j.H_{i,j} \end{pmatrix}$$

    - Construction de la matrice variance S définie par :

$$S = P_0\,AA^T + P_1\,BB^T$$ où X^T est la transposée d'une matrice X ;

$$A = (Moy_0 - Moy) \text{ et } B = (Moy_1 - Moy)$$

    - Calcul de la valeur de la trace de la matrice S et mise à jour de la valeur de la trace maximale. On rappelle que la trace d'une matrice correspond à la somme des valeurs des éléments de sa diagonale principale.

Travail demandé

N.B : Dans la suite, les fonctions demandées seront écrites en langage Python, en utilisant impérativement la nomenclature donnée par le tableau suivant.

Nom	Type	Description
M	*numpy.ndarray*	Matrice d'entiers (entre 0 et 255) représentant une image
V	*numpy.ndarray*	Matrice d'entiers contenant les moyennes des voisinages de ***M***
n, m	*int*	Respectivement nombre de lignes et de colonnes de ***M*** et ***V***
H	*numpy.ndarray*	Matrice histogramme d'ordre 256 correspondant à ***M*** et ***V***

1. Ecrire la fonction, nommée **GenererMat**, qui prend en paramètres n et m et retourne une matrice M remplie par des valeurs entières aléatoires comprises entre 0 et 255.
2. Ecrire la fonction, nommée **MoyVoisinage**, qui prend en paramètre M et retourne V.
3. Ecrire la fonction, nommée **NbPixels**, qui prend en paramètres M, V et deux entiers i et j. Elle retourne le nombre d'éléments de M ayant $M_{a,b} = i$ et $V_{a,b} = j$ (avec $0 \leq a \leq n-1$ et $0 \leq b \leq m-1$).
4. Ecrire la fonction, nommée **Histogramme**, qui prend en paramètres M et V et retourne la matrice H.
5. Ecrire la fonction, nommée **Sigma**, qui prend en paramètres H, ***binf1***, ***binf2***, ***bsup1***, ***bsup2*** et ***comp*** (paramètre ayant la valeur par défaut 0), calcule et retourne :
   - $\sum_{i=\text{binf}_1}^{\text{bsup}_1} \sum_{j=\text{binf}_2}^{\text{bsup}_2} i.H_{i,j}$ si ***comp*** égal à 0 ;
   - $\sum_{i=\text{binf}_1}^{\text{bsup}_1} \sum_{j=\text{binf}_2}^{\text{bsup}_2} j.H_{i,j}$ si ***comp*** égal à 1.
6. Ecrire la fonction, nommée **Seuillage**, qui à partir du paramètre H, retourne le couple optimal (s,t) correspondant au maximum de toutes les valeurs des traces des matrices S.

Partie II

Une région est un ensemble de pixels représentant une partie ou la totalité d'une image.
L'exemple suivant illustre une image à 3 lignes et 5 colonnes où les cases hachurées forment une région.

	0	1	2	3	4
0	128	64	128	32	32
1	64	**255**	**0**	**64**	128
2	0	0	**255**	**64**	54

pixel de la région de coordonnées (1, 3) avec un ton de gris = 64

Figure 1 : Exemple de région d'une image et ses pixels

Dans le but de binariser une région d'une image, dans cette partie, on propose d'implémenter les classes : Pixel et Region.

Description des classes

- Classe **Pixel** :
  - Attributs :
    - **posi** et **posj**, des entiers représentant les coordonnées du pixel ;
    - **ton**, entier entre 0 et 255, représentant le niveau de gris du pixel.
  - Méthodes :
    - **__init__(...)** : permet d'initialiser un pixel ;
    - **__str__(...)** : permet d'afficher un pixel selon le format `Pixel<posi,posj,ton>`.

Exemple : px = Pixel(1, 3, 64) est un objet, instance de la classe Pixel.

- Classe **Region** :
  - Attributs :
    - **label**, chaîne de caractères, désignant la région ;
    - **dict_pixel**, un dictionnaire où :
      - chaque clé représente un ton de gris ;
      - chaque valeur représente la liste des coordonnées des pixels ayant ce ton de gris. *Exemple* : Le dictionnaire associé à l'instance de la classe Region donnée par l'exemple de la Figure 1 est :
        `dict_pixel ={255 :[(1,1),(2,2)], 64 :[(1,3),(2,3)], 0:[(1,2)]}.`
  - Méthodes :
    - **__init__(...)** : permet l'initialisation des attributs de la classe Region et dont le script Python est donné par :

```
def __init__(self , lab):
    self.label = lab
    self.dict_pixel = dict()
```

    - **__len__(...)** : retourne la taille de la région, en nombre de pixels. Pour l'exemple de région de la Figure 1, cette méthode retourne 5.
    - **__call__(...)** : retourne la liste des coordonnées des pixels de la région ayant un ton de gris *t*. Dans le cas où ce ton n'existe pas dans la région, cette méthode retourne `None`. Pour l'exemple de la Figure 1 et le ton 64, cette méthode retourne la liste `[(1,3),(2,3)]`.
    - **__contains__(...)** : retourne `True` si un pixel *px* appartient à la région et `False` sinon.
    - **ajouter_pixel(...)** : ajoute un pixel *px* à la région.
    - **supprimer_pixel(...)** : supprime un pixel *px* de la région.
    - **binariser_reg(...)** : retourne, à partir d'un ton *t*, une nouvelle région à deux tons 0 et 255 tels que le ton 0 est associé aux pixels ayant un ton strictement inférieur à *t* et le ton 255 est associé aux autres. Le label de la nouvelle région est celui de la région d'origine post-fixé par la chaîne 'bin'.

Travail demandé

Dans la suite, les réponses seront écrites en langage Python, en se basant sur les descriptions des classes déjà données.

1. Construire la classe **Pixel**.

Pour la classe **Region** :

2. Ecrire la méthode **__len__**.
3. Ecrire la méthode **__call__**.
4. Ecrire la méthode **__contains__**.
5. Ecrire la méthode **ajouter_pixel**.
6. Ecrire la méthode **supprimer_pixel**.
7. Ecrire la méthode **binariser_reg**.

8. En supposant disposer d'une matrice ***Im*** de type `numpy.ndarray` représentant une image en niveaux de gris, écrire les instructions Python permettant de :
   - créer, à partir de ***Im***, une instance de la classe Region, notée ***Reg*** et ayant le label 'Paysage' ;
   - binariser la région ***Reg*** à partir d'un ton de gris, noté ***seuil***, supposé déjà calculé.

PROBLEME 2

Soit le schéma relationnel de la base de données des images 'BDDImage.db', défini par les trois relations suivantes :

- **Image** (IdImage, nomF, extF, largeur, hauteur)

  La table Image contient toutes les images actuellement disponibles dans la base.
  - IdImage : identifiant de l'image (chaîne de caractères), clé primaire.
  - nomF : nom du fichier correspondant à l'image (chaîne de caractères).
  - extF : extension du fichier correspondant à l'image (chaîne de caractères).
  - hauteur : hauteur de l'image en nombre de pixels (entier).
  - largeur : largeur de l'image en nombre de pixels (entier).

- **Region** (IdRegion, label, IdImage)

  La table Region contient toutes les régions associées aux images sachant qu'une région est une partie d'une image.
  - IdRegion : identifiant de la région (chaîne de caractères), clé primaire.
  - label : désignation de la région (chaîne de caractères).
  - IdImage : identifiant de l'image (chaîne de caractères) à laquelle appartient la région identifiée par IdRegion, clé étrangère.

- **Pixel** (X, Y, IdRegion, rouge, vert, bleu).

  La table Pixel contient tous les pixels couleur associés aux régions.
  - X : position au niveau des lignes dans l'image (entier).
  - Y : position au niveau des colonnes dans l'image (entier).
  - IdRegion : identifiant de la région à laquelle appartient le pixel de positions X et Y (chaîne de caractères), clé étrangère.
  - rouge : valeur de la couleur rouge, (entier entre 0 et 255).
  - vert : valeur de la couleur vert, (entier entre 0 et 255).
  - bleu : valeur de la couleur bleu, (entier entre 0 et 255).

  Le triplet (X, Y, IdRegion) constitue la clé primaire de la table Pixel.

Dans la suite, on suppose que les trois tables de 'BDDImage.db' sont déjà créées et remplies.

Partie I

Soit le script python suivant :

```
import sqlite3
connexion = sqlite3.connect('BDDImage.db')
curseur = connexion.cursor()
   ...
curseur.close()
connexion.close()
```

Dans la suite, `connexion` et `curseur` seront utilisées par les fonctions comme des variables globales.

1. Ecrire la fonction Python, nommée **GetData**, qui, à partir d'une table ***T***, détermine et retourne la liste de toutes les informations des lignes de cette table.
2. Ecrire la fonction Python, nommée **RechercheImages**, qui, à partir de deux entiers ***minK*** et ***maxK***:
   - détermine la liste ***L*** de toutes les informations des images de la table Image ;
   - détermine, à partir de ***L***, la liste des identifiants des images dont la taille (en Kilo octets) est comprise entre ***minK*** et ***maxK***;
   - retourne la liste résultante.

   N.B : La valeur de chaque couleur (entier entre 0 et 255) est codée sur 1 octet.

Partie II

Donner les requêtes SQL permettant de :

3. Déterminer les noms et les dimensions en nombre de pixels des images ayant l'extension 'png'. Le résultat doit être classé par ordre décroissant selon la dimension.
4. Déterminer le nombre de pixels par région.
5. Déterminer les identifiants des images comportant exactement deux régions.
6. Déterminer les positions des pixels de dominance rouge de la région ayant le label 'Cercle'. Un pixel de dominance rouge est un pixel dont la valeur de la couleur rouge est supérieure à celles des deux autres couleurs.
7. Sachant qu'un pixel bleu a la valeur du triplet (rouge, vert, bleu) = (0, 0, 255), modifier en bleu tous les pixels de la région d'identifiant 'MER102'.

Partie III

Ecrire en algèbre relationnelle les expressions suivantes :

8. Déterminer les positions des pixels de la région ayant le label 'Cercle'.
9. Déterminer les noms des fichiers des images n'ayant aucune région associée.

REPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur,
de la Recherche Scientifique

Concours Nationaux d'Entrée
aux Cycles de Formation d'Ingénieurs
Session 2018

الجمهورية التونسية
وزارة التعليم العالي والبحث العلمي

المناظرات الوطنية للدخول
إلى مراحل تكوين المهندسين
دورة 2018

Altérative de Correction
Concours Mathématique et Physique, Physique et Chimie et Technologie
Epreuve d'Informatique

PROBLEME 1

Partie I

```
import numpy as np
from random import randint
```

1.

```
def GenererMat(n,m):
    M=np.ndarray(shape=(n,m),dtype=int)
    #ou M=np.empty((n,m),dtype=int)
    for i in range(n):
        for j in range(m):
            M[i,j]= randint(0,255)
             # ou encore np.random.randint (0,256)
    return M
```

2.

```
def MoyVoisinage(M):
    V= M.copy()
    (n,m)=M.shape #ou (n,m)=np.shape(M)
    for i in range(1,n-1):
        for j in range(1,m-1):
            V[i,j]=sum([M[k,l] for k in range(i-1,i+2) \
                    for l in range(j-1,j+2)]) // 9
    return V
```

3.

```
def NbPixels(M,V,i,j):
    (n,m)=M.shape #ou (n,m)=np.shape(M)
    nb=0
    for a in range(n):
        for b in range(m):
            if M[a,b]==i and V[a,b]==j:
                nb+=1
    return nb
```

4.

```
def Histogramme (M,V):
    H=np.zeros((256,256))
    for i in range(256):
        for j in range(256):
            H[i,j]=NbPixels(M,V,i,j)
    return H
```

5.

```
def Sigma(H,binf1,binf2,bsup1,bsup2,comp1=0):
    if not comp1:
        return sum([i*H[i,j] for i in range(binf1,bsup1+1) \
              for j in range(binf2,bsup2+1)])
    else:
        return sum([j*H[i,j] for i in range(binf1,bsup1+1) \
              for j in range(binf2,bsup2+1)])
```

6.

```
def seuillage(H):

    Moy=np.array([[Sigma(H,0,0,256,256)],[Sigma(H,0,0,256,256,1)]
    ])
    varmax=-1
    for s in range(255):
        for t in range(255):
            c=256*256
            P0=sum([H[i,j] for i in range(s+1)\
                   for j in range(t+1)])/c
            P1=sum([H[i,j] for i in range(s+1,256) \
                  for j in range(t+1,256)])/c
            if P0!=0.0 and  P1!=0.0:
                Moy0=(1/P0)*np.array([[Sigma(H,0,0,s+1,t+1)],\
                  [Sigma(H,0,0,s+1,t+1,1)]])
                Moy1=(1/P1)*np.array([[Sigma(H,s+1,t+1,255,255)],
                \
                  [Sigma(H,s+1,t+1,255,255,1)]])
                A=Moy0-Moy
                B=Moy1-Moy
                S=P0*np.dot(A,np.transpose(A))+ \
                  P1*np.dot(B,np.transpose(B))
                var=np.trace(S)
                if varmax<var:
                    varmax=var
                    stMax=(s,t)
    return stMax
```

Partie II

```
#Réponse question 1
class Pixel:
    def __init__(self,a,b,c):
        self.posi=a
        self.posj=b
        self.ton=c
    def __str__(self):
        return   'Pixel   <{},{},{}>'.format(self.posi,self.posj,\
                  self.ton)
        # ou encore return
        #'Pixel <'+str(self.posi)+','+str(self.posj)+ \
            ','+str(self.ton)+'>'

class Region:
    def __init__(self,lab):
        self.label=lab
        self.dict_pixel={}

#Réponse question 2
    def __len__(self):
        nb=0
        for ton in self.dict_pixel: # ou self.dict_pixel.keys()
            nb+=len(self.dict_pixel[ton])
        return nb

#Réponse question 3
    def __call__(self,ton):
       try :
            assert(ton in self.dict_pixel)
            return self.dict_pixel[ton]
       except:
            return None

#Réponse question 4
    def __contains__(self,px):
        #version 1
        if px.ton in self.dict_pixel:
            if (px.posi,px.posj) in self.dict_pixel[px.ton]:
                return True
            else : return False
        else : return False

#Réponse question 5
    def ajouter_pixel(self,px):
        #version1
        if px.ton in self.dict_pixel:
            self.dict_pixel[px.ton].append((px.posi,px.posj))
        else:
            self.dict_pixel[px.ton]=[(px.posi,px.posj)]
```

```
#Réponse question 6
    def supprimer_pixel(self,px):
        if px.ton in self.dict_pixel:
            if len(self(px.ton)) == 1:
                self.dict_pixel.pop(px.ton)
            else :
                self.dict_pixel[px.ton].remove((px.posi,px.posj))
        else :  print("le pixel n'existe pas")

#Réponse question 7
    def binariser_reg(self,seuil):
        other=Region(self.label+'bin')
        other.dict_pixel[0]=[]
        other.dict_pixel[255]=[]
        for i in self.dict_pixel:
            if i<= seuil:
                other.dict_pixel[0].extend(self.dict_pixel[i])
            else:
                other.dict_pixel[255].extend(self.dict_pixel[i])
        return other

#Réponse question 8
Reg=Region('Paysage')

for i in range(Im.shape[0]):
    for j in range(Im.shape[1]):
        px=Pixel(i,j,Im[i,j])
        Reg.ajouter_pixel(px)

Regbin=Reg.binariser_reg(seuil)
```

PROBLEME 2

Partie I

1.

```
def getData(T) :
    req='select * from ' + T + ';'
    curseur.execute(req)
    l=curseur.fetchall()
    return l
```

2.

```
def rechercheImages(mink , maxk):
    l=getData('image')
    lident=[]
    for e in l:
        taille=3.0*e[3]*e[4]/1024
        if taille<=maxk and taille>=mink :
            lident.append(e[0])
    return lident
```

Partie II

3.

```
SELECT  nomF , hauteur * largeur AS d
FROM  Image
WHERE  extF = 'png'
ORDER BY  d  DESC ;
```

4.

```
SELECT  IdRegion , count (*)
FROM  Pixel
GROUP BY  IdRegion ;
```

5.

```
SELECT  IdImage , count (IdRegion) c
FROM  Image
GROUP BY  IdImage
HAVING  c = 2 ;
```

6.

```
SELECT  Pixel.X , Pixel.Y
FROM  Region , Pixel
WHERE  Region.IdRegion = Pixel.IdRegion AND  Region.label = 'cercle'
AND Pixel.rouge > Pixel.vert  AND  Pixel.rouge > pixel.bleu ;
```

7.

```
UPDATE  Pixel
SET  rouge = 0 , vert = 0 , bleu = 255
WHERE  IdRegion = 'MER102' ;
```

Partie III (5 points)

8.

$$\pi\,(X\,,Y\,)\ \left(\sigma(label\ =\ "Cercle")(Pixel\,)\ \underset{IdRegion}{\propto}\ (Region)\right)$$

9.

$$\pi\,(nomF\,)\ \ (Image) - \pi\,(nomF\,)\ \left((Image\,)\ \underset{IdImage}{\propto}\ (Region)\right)$$